06 POINTERS AND REFERENCES • THEORY

04 memory hazards and smart pointers

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

04 — Memory Hazards (`new` / `delete` Pitfalls) and Modern Smart Pointers

1. The Four Classic Pointer & Memory Hazards

A. Wild Pointers (Uninitialized Pointers)

A pointer declared without initialization contains arbitrary garbage bits left on the stack:

```cpp
int* p;      // Wild pointer! Contains indeterminate address
*p = 10;     // UNDEFINED BEHAVIOR (dereferencing indeterminate address)
```

Prevention: Always initialize every pointer at declaration (`int* p = nullptr;` or `int* p = &x;`).

B. Dangling Pointers

A pointer that once pointed to a valid object, but the object's lifetime has ended (either because a local stack variable went out of scope, or because `delete` was called on heap memory):

```cpp
int* p = new int(42);
delete p;    // Memory is returned to free store, but p still holds old address!
// p is now a DANGLING POINTER
*p = 10;     // UNDEFINED BEHAVIOR (use-after-free)
p = nullptr; // Good practice with raw pointers: reset immediately after delete
```

Another classic dangling pointer mistake is returning the address of a local stack variable from a function (`return &local_var;`).

C. Memory Leaks

Allocating dynamic memory with `new` and losing the pointer address (by overwriting the pointer or letting it go out of scope) without calling `delete`:

```cpp
void leak_example() {
    int* p = new int(100);
    // Function returns without 'delete p;' → 4 bytes leaked permanently until process exit
}
```

D. Double Deletion

Calling `delete` twice on the same heap address corrupts the heap allocator's internal free list and results in **Undefined Behavior**:

```cpp
int* p = new int(50);
delete p;
delete p; // UNDEFINED BEHAVIOR (Double Free)
```

---

2. Classroom Approach vs. Modern C++ Smart Pointers (`<memory>`)

Step 1: Classroom Raw Pointer Approach

In introductory lab exams, dynamic object creation is taught using `new` and `delete`:

```cpp
person* p = new person;
p->input(21);
p->display();
delete p;
```

Limitation: If an exception is thrown or an early `return` happens before `delete p;`, the `person` object is leaked.

Step 2: Modern C++ Approach (`std::unique_ptr` , `std::shared_ptr` , `std::weak_ptr`)

Modern C++ (C++11/14/17/20/23) wraps raw pointers inside RAII class templates defined in `<memory>` that automatically `delete` the managed object in their destructor:

1. `std::unique_ptr<T>` **(Exclusive Ownership — C++11,** `std::make_unique` **in C++14)**:
   - Exactly **one** `unique_ptr` owns the heap object.
   - Cannot be copied (`p2 = p1` is a compilation error), only moved (`std::move(p1)`).
   - Zero size or speed overhead compared to a raw pointer.

   ```cpp
   auto p = std::make_unique<person>();
   p->input(21);
   p->display();
   // Automatically deleted when p goes out of scope!
   ```

2. `std::shared_ptr<T>` **(Shared Reference-Counted Ownership — C++11)**:
   - Multiple `shared_ptr` instances can co-own the same heap object.
   - Maintains an atomic **reference count** in a control block.
   - The object is destroyed only when the *last* `shared_ptr` owning it is destroyed or reset.
3. `std::weak_ptr<T>` **(Non-Owning Observer — C++11)**:

- Observes an object managed by a `shared_ptr` **without** incrementing its strong reference count.
- Used to break circular reference cycles (`A` holds `shared_ptr<B>` and `B` holds `weak_ptr<A>`).
- Must call `.lock()` to obtain a temporary `shared_ptr` before accessing the object.